



MODUL PERKULIAHAN

OPTIMALISASI & OTOMASI

Kode Mata kuliah P132520004

Modul 4

Preprocessing Dasar: Pembersihan

Abstrak

Pertemuan ini membahas tahap fundamental preprocessing data sensor time series pada konteks predictive

Sub - CPMK

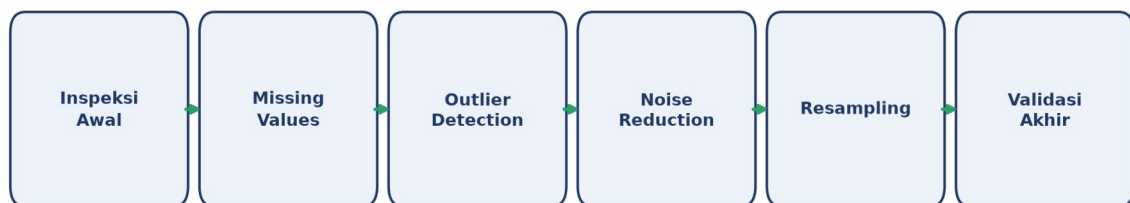
Sub-CPMK 2.1 – Mampu menganalisis dampak sosial dan lingkungan dari sistem otomatisasi

Disusun Oleh :
Dedik Romahadi, ST., M.Sc

🔗 Modul Interaktif: <https://dedik-romahadi.github.io/Mechanical-Engineering-Courses/Optimalisasi-dan-Automasi/Modul/Modul-4.html>. Pada layar masuk, pilih tombol “👁 Mode Preview (Tanpa Login)” untuk melihat modul lengkap (animasi, kalkulator interaktif, editor kode Python langsung di browser) tanpa perlu akun. Soal pada Mode Preview bersifat tampilan saja; jawaban benar/salah dan poin tidak aktif.

PENDAHULUAN

Pertemuan keempat ini membahas tahap fundamental preprocessing data sensor time series pada konteks predictive maintenance. Prinsip "Garbage In, Garbage Out" berlaku mutlak: keputusan preprocessing yang keliru dapat menghapus sinyal peringatan dini kerusakan yang justru ingin dideteksi, sehingga berdampak pada keandalan operasional dan keselamatan kerja secara luas. Gambar 1 menunjukkan pipeline enam tahap pembersihan data yang dibahas pada pertemuan ini.



Gambar 1. Pipeline pembersihan data time series enam tahap: Inspeksi Awal, Missing Values, Outlier Detection, Noise Reduction, Resampling, dan Validasi Akhir.

Preprocessing memakan sekitar 60-80% waktu kerja data scientist dalam praktiknya. Contoh skala data: accelerometer 200 Hz yang direkam selama 8 jam menghasilkan 5,76 juta titik data yang harus diperiksa dan dibersihkan sebelum dianalisis. Materi ditutup dengan implementasi Python di Jupyter Notebook, tugas terstruktur, dan forum diskusi studi kasus.

Capaian Pembelajaran (Sub-CPMK)

- **Sub-CPMK 2.1:** Mampu menganalisis dampak sosial dan lingkungan dari sistem otomasi, yaitu memahami bagaimana keputusan preprocessing data (pembersihan missing value, outlier, dan noise) memengaruhi keandalan sistem monitoring yang pada akhirnya berdampak pada keselamatan operasional dan lingkungan kerja (CPMK 2).
- Setelah pertemuan ini mahasiswa dapat: menjelaskan pipeline pembersihan data time series enam tahap; mendeteksi dan menangani missing values dengan teknik yang sesuai; mendeteksi outlier dengan Z-score dan IQR tanpa membuang sinyal

kerusakan dini; menerapkan noise reduction yang tepat; serta melakukan resampling dan normalisasi data multi-sensor.

MISSING VALUES: DETEKSI & PENANGANAN

Missing values dideteksi dengan `df.isnull().sum()` untuk menghitung jumlah nilai kosong per kolom, `df.isna().mean()*100` untuk persentasenya, atau `df[df.isnull().any(axis=1)]` untuk menampilkan baris yang mengandung nilai kosong.

- **Drop:** menghapus baris yang mengandung NaN (`df.dropna()`); cocok bila persentase missing kecil (<5%) dan tersebar acak.
- **Forward/Backward Fill:** mengisi dengan nilai observasi terakhir (`df.fillna(method='ffill')`); cocok untuk data yang lambat berubah.
- **Mean/Median Fill:** mengisi dengan rata-rata atau median kolom (`df.fillna(df.mean())`); sederhana tetapi menumpulkan variabilitas data.
- **Interpolasi:** mengisi berdasarkan interpolasi nilai tetangga (`df.interpolate(method='time')`); umumnya menjadi pilihan terbaik untuk time series numerik.

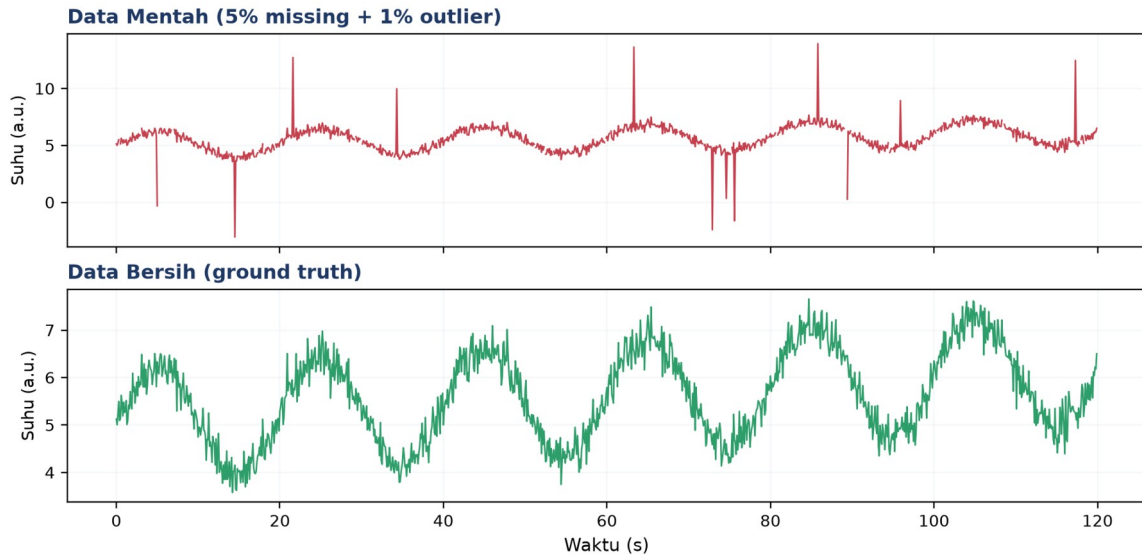
Tabel 1 merangkum lima pola missing values yang umum dijumpai beserta karakteristik dan teknik penanganan yang disarankan.

Tabel 1. Pola missing values pada data sensor dan teknik penanganan yang disarankan.

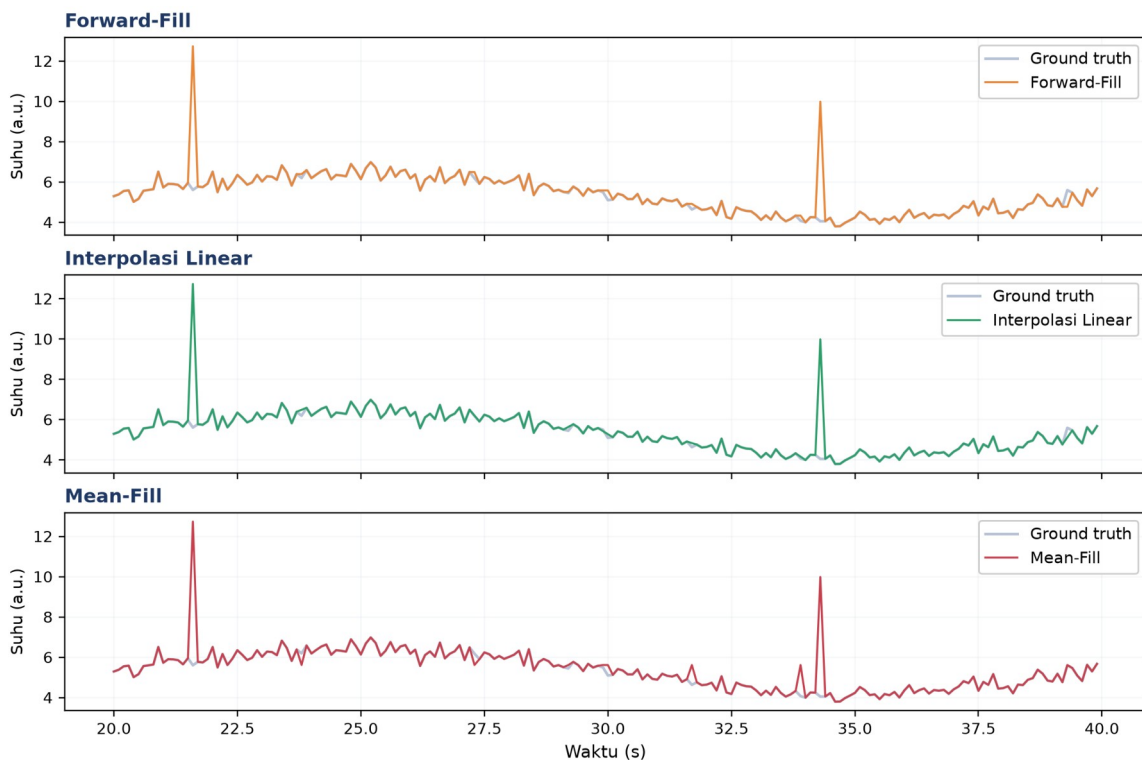
Pola Missing	Karakteristik	Teknik Disarankan
MCAR (Acak Sepenuhnya)	Hilang tersebar tanpa pola	Drop atau Mean Fill
MAR (Acak Bersyarat)	Hilang berkorelasi dengan kolom lain	Interpolasi atau model-based imputation
MNAR (Tidak Acak)	Hilang ketika sensor jenuh/di luar batas	Tandai dengan flag, jangan diisi sembarangan
Gap waktu pendek (<5 sampel)	Komunikasi sesaat terputus	Forward-fill atau interpolasi linear
Gap waktu panjang (>1 menit)	Mesin off/sensor maintenance	Tandai segmen, hindari interpolasi paksa

Mengapa Klasifikasi Pola Missing Penting: Penting dibedakan antara MCAR, MAR, dan MNAR karena kesalahan klasifikasi pola missing values dapat menghasilkan bias sistematis pada analisis lanjutan. Contoh nyata MNAR pada sensor industri: accelerometer yang mencatat nilai kosong justru saat amplitudo getaran melampaui rentang pengukuran (sensor saturasi/clipping), sehingga dalam kasus ini, data yang hilang JUSTRU berkorelasi dengan kondisi paling kritis yang ingin dideteksi. Mengisi nilai kosong tersebut dengan mean atau interpolasi akan secara sistematis menyembunyikan kejadian ekstrem yang

sebenarnya paling penting untuk sistem predictive maintenance, sehingga pendekatan yang benar adalah menandainya dengan flag khusus (mis. ``sensor_saturated=True``) alih-alih mengisi nilainya sama sekali. Skemanya diperlihatkan pada Gambar 2.



Gambar 2. Perbandingan data mentah (5% missing + 1% outlier) dengan data bersih (ground truth).



Gambar 3. Perbandingan tiga teknik imputasi: Forward-Fill, Interpolasi Linear, dan Mean-Fill pada segmen data yang sama.

Gambar 3 memperlihatkan interpolasi linear mengikuti bentuk data ground truth paling dekat, forward-fill menciptakan efek "tangga" yang kurang halus, sementara mean-fill paling menyimpang karena mengabaikan tren lokal sinyal.

Peringatan: Mengisi missing values saat mesin sedang shutdown dengan mean atau forward-fill membuat model seolah-olah melihat mesin terus berjalan normal, padahal sesungguhnya tidak beroperasi, sehingga data menjadi menyesatkan bagi analisis lanjutan.

OUTLIER DETECTION & PENANGANAN

Dua metode deteksi outlier yang paling umum dipakai adalah Z-score dan IQR (Interquartile Range). Hubungan ini dirangkum dalam Persamaan (1).

$$\text{Z-score: } z = (x - \mu) / \sigma, \text{ outlier jika } |z| > 3 \quad \text{IQR: outlier jika } x < Q1 - 1,5 \cdot \text{IQR atau } x > Q3 + 1,5 \cdot \text{IQR} \quad (1)$$

- **Removal:** menghapus titik yang terdeteksi sebagai outlier (`df[z<3]`).
- **Capping/Winsorizing:** membatasi nilai ekstrem ke persentil tertentu tanpa menghapus titik data (`np.clip(x, q01, q99)`).
- **Flag & Investigate:** menandai titik sebagai outlier tanpa mengubah nilainya (`df['is_outlier']=(z>3)`), direkomendasikan khusus untuk monitoring kondisi mesin agar sinyal kerusakan tidak hilang.
- **Imputasi:** mengganti nilai outlier dengan median lokal (`x[outlier]=median(x)`).

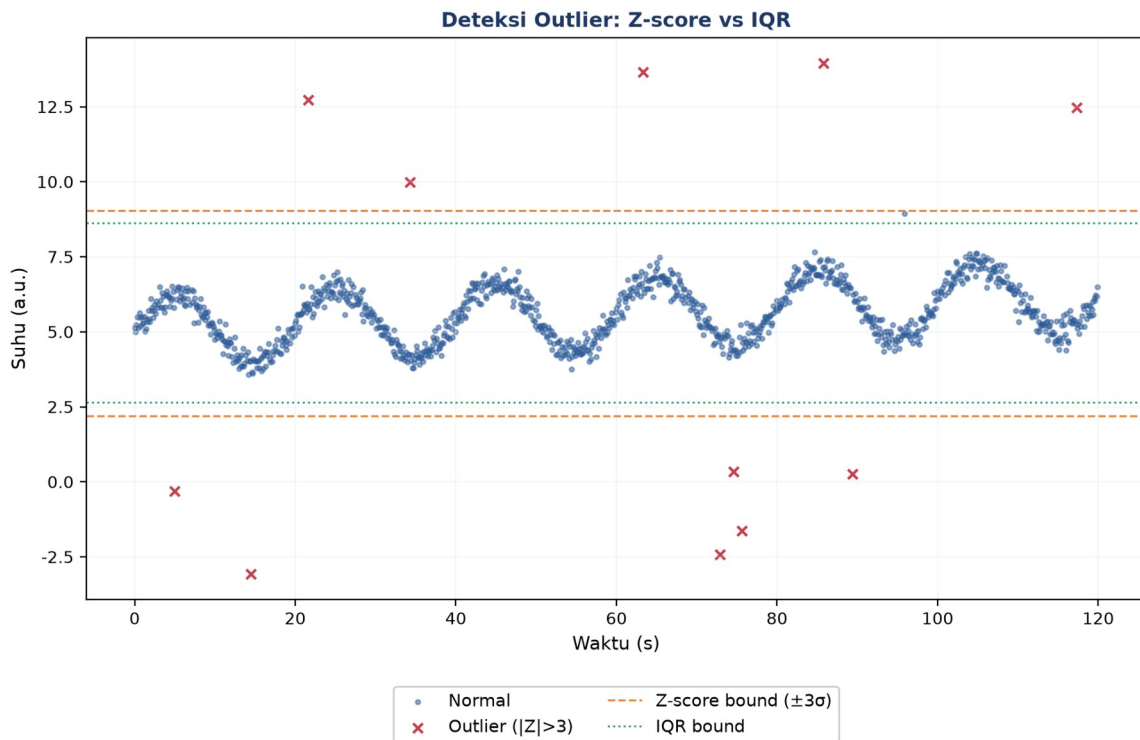
Tabel 2 membandingkan lima metode deteksi outlier dari sisi asumsi distribusi, keunggulan, dan keterbatasannya.

Tabel 2. Perbandingan lima metode deteksi outlier terhadap asumsi distribusi, keunggulan, dan keterbatasan.

Metode	Asumsi Distribusi	Keunggulan	Keterbatasan
Z-Score	Mendekati normal (Gaussian)	Cepat, intuitif, mudah diinterpretasi	Sensitif terhadap outlier itu sendiri
IQR	Bebas asumsi distribusi	Robust, tidak dipengaruhi outlier ekstrem	Lebih konservatif, kadang melewatkan outlier moderat
Modified Z-Score (MAD)	Bebas asumsi distribusi	Robust dengan skala mirip Z-score (threshold 3,5)	Perlu perhitungan tambahan
Rolling Z-Score	Stasioner dalam window	Cocok untuk time series dengan tren/musiman	Pemilihan window size krusial
Isolation Forest	Bebas asumsi (model ML)	Multi-variat, menangkap outlier kompleks	Black-box, butuh tuning hyperparameter

Kapan Isolation Forest Diperlukan: Isolation Forest layak dipertimbangkan ketika outlier bersifat multivariat, yaitu tidak terdeteksi bila tiap sensor dianalisis satu per satu, tetapi menjadi jelas saat kombinasi beberapa sensor diperiksa bersamaan. Contohnya, suhu

bearing 60°C dan RPM 1500 masing-masing normal bila dilihat terpisah, namun kombinasi keduanya (suhu tinggi pada RPM rendah) justru mengindikasikan gesekan berlebih akibat pelumasan yang buruk, yaitu pola yang hanya terlihat pada ruang fitur multi-dimensi, bukan pada distribusi univariat Z-score atau IQR sederhana. Gambar 4 merangkum alurnya secara visual.



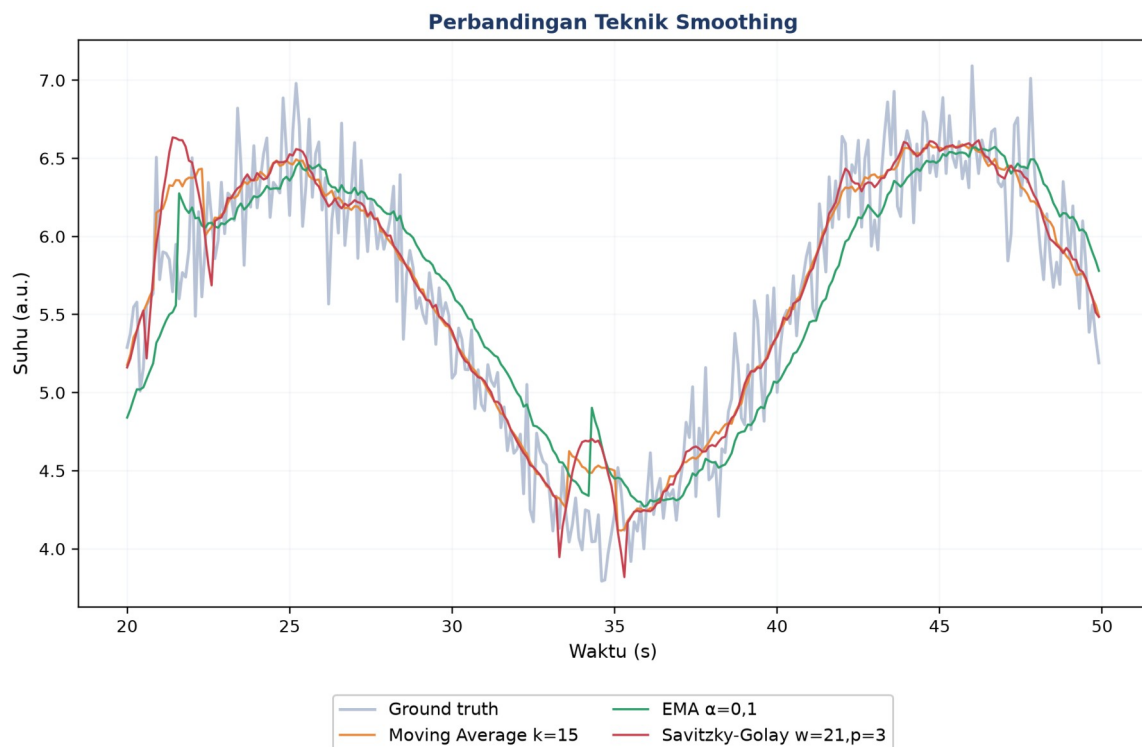
Gambar 4. Deteksi outlier dengan batas Z-score (garis putus-putus oranye) dan IQR (garis titik-titik hijau); titik merah ditandai sebagai outlier.

Studi kasus: Sebuah bearing pompa sentrifugal dengan amplitudo getaran rata-rata 2 mm/s tiba-tiba melonjak ke 25 mm/s selama 0,3 detik, menghasilkan Z-score sekitar 8 yang jelas terdeteksi sebagai outlier oleh algoritma standar. Namun lonjakan ini sebenarnya adalah impulse dari bearing yang mulai retak; menghapusnya sebagai outlier berarti membuang sinyal peringatan dini yang justru paling berharga.

NOISE REDUCTION: SMOOTHING & FILTER

Empat teknik smoothing yang umum dipakai: Moving Average (`df.rolling(w).mean()`, window besar berarti lebih halus tetapi delay lebih besar), Exponential Moving Average (`df.ewm(alpha=0.3).mean()`, memberi bobot lebih besar pada data terbaru), Savitzky-Golay Filter (`savgol_filter(x,51,3)`, mempertahankan bentuk puncak lebih baik), dan Low-Pass Filter (`scipy.signal.butter(N, fc)`, Butterworth/FIR untuk filtering yang lebih presisi).

Kapan Memilih Butterworth: Low-Pass Filter Butterworth layak menjadi pilihan default saat presisi kontrol frekuensi cutoff menjadi prioritas, karena parameternya (orde N , frekuensi cutoff f_c) langsung dapat diterjemahkan ke spesifikasi respons frekuensi yang diinginkan, berbeda dengan Moving Average yang parameter window-nya (k) hanya berhubungan tidak langsung dengan frekuensi cutoff efektif melalui hubungan $f_{\text{cutoff}} \approx f_s/k$. Pada scipy, `'butter(N, fc, fs=fs)'` mengembalikan koefisien filter yang kemudian diterapkan via `'filtfilt'` (zero-phase, non-causal) untuk analisis offline, atau `'filter'` (causal, ada phase lag) untuk aplikasi real-time. Gambar 5 mengilustrasikan gagasan ini.



Gambar 5. Perbandingan Moving Average, EMA, dan Savitzky-Golay pada segmen data yang sama.

Cara memilih window size: Untuk mempertahankan komponen 1 Hz pada data yang di-sampling 100 Hz, sebaiknya window moving average tidak melebihi 100 sampel; window yang lebih besar akan meredam komponen frekuensi tersebut secara tidak sengaja.

Peringatan: Smoothing yang terlalu kuat dapat menyamarkan impulse kerusakan bearing (buruk untuk deteksi kerusakan), meskipun bermanfaat untuk melihat tren suhu jangka panjang. Pemilihan intensitas smoothing harus disesuaikan dengan tujuan analisis.

ANALOGI KEHIDUPAN SEHARI-HARI

- **Bersih-bersih Dapur sebelum Memasak:** mempersiapkan bahan sebelum memasak, mencuci dan memotong, adalah analogi langsung dari tahap preprocessing sebelum data siap dianalisis.
- **Sambungan Telepon Terputus Sesaat:** otak manusia otomatis mengisi kata yang hilang saat sambungan telepon terputus sesaat, mirip interpolasi; gap pendek aman ditebak, tetapi gap panjang berbahaya untuk ditebak sembarangan.
- **Filter Foto Smartphone:** filter noise reduction yang terlalu kuat pada foto membuat wajah tampak seperti plastik, sama seperti smoothing berlebihan yang menghilangkan detail bermakna pada sinyal.
- **Termometer yang Kadang Salah Baca:** membedakan termometer yang salah baca (outlier error) dari suhu ekstrem nyata seperti gelombang panas adalah persoalan yang identik dengan membedakan outlier sensor dari kejadian fisik nyata.
- **Membandingkan Nilai Ujian Antar Sekolah:** membandingkan nilai ujian antar sekolah dengan skala penilaian berbeda membutuhkan normalisasi/standardisasi, sama seperti menyetarakan skala fitur multi-sensor sebelum dianalisis bersama. Bentuk ringkasnya dituliskan pada Persamaan (2).

RESAMPLING, NORMALISASI & VALIDASI AKHIR

$$\text{Min-Max: } x_{\text{norm}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}}) \quad \text{Z-score: } x_{\text{std}} = (x - \mu) / \sigma \quad (2)$$

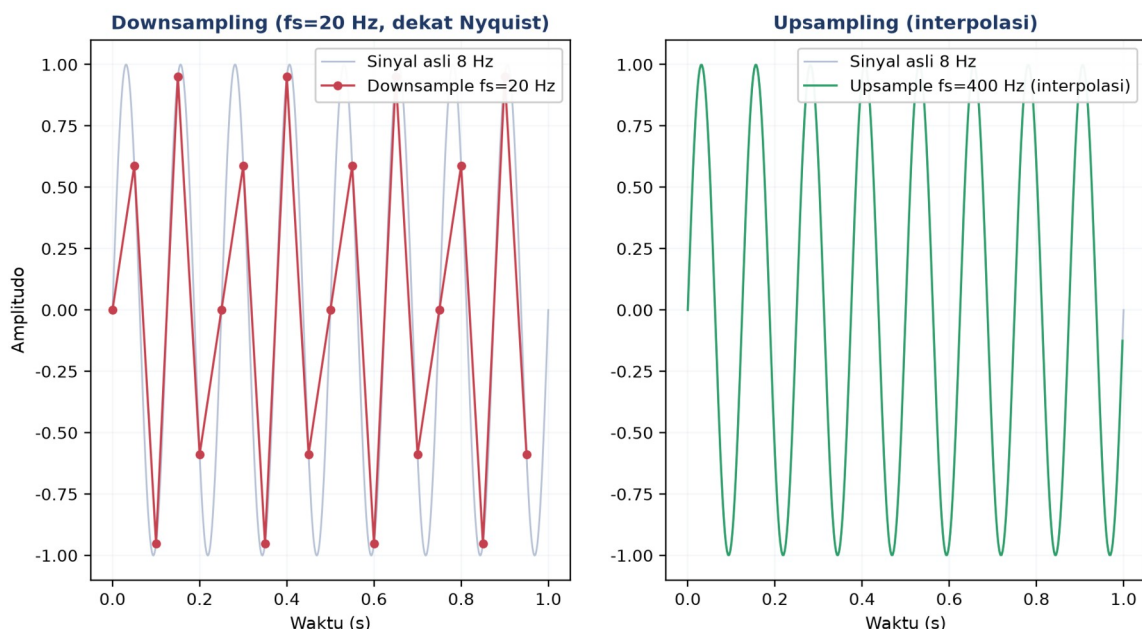
- **Downsampling/Aggregation:** agregasi data ke sampling rate lebih rendah (`df.resample('1S').mean()`); harus hati-hati terhadap potensi aliasing.
- **Upsampling/Interpolation:** interpolasi data ke sampling rate lebih tinggi (`df.resample('10S').interpolate()`).
- **Min-Max Scaling:** menyamakan rentang ke [0,1] (`MinMaxScaler()`); cocok untuk algoritma seperti k-NN dan neural network.
- **Z-Score Standardization:** menyamakan mean=0, std=1 (`StandardScaler()`); cocok untuk PCA dan regresi linear, lebih robust terhadap outlier dibanding Min-Max.

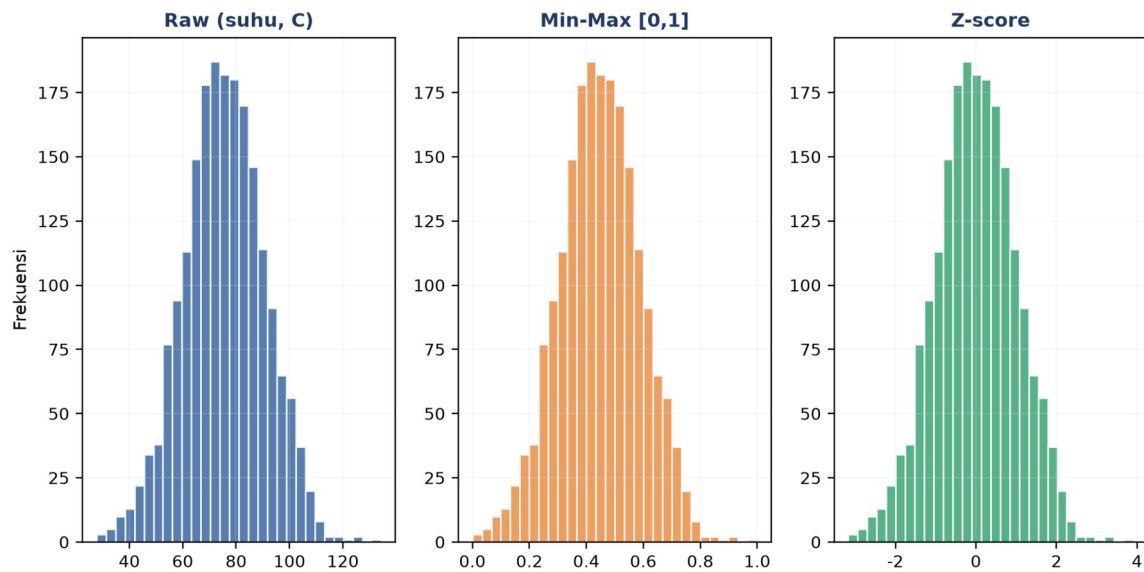
Tabel 3 merangkum konteks aplikasi dan teknik resampling/normalisasi yang sesuai.

Tabel 3. Konteks aplikasi dan teknik resampling/normalisasi yang sesuai.

Konteks	Teknik Disarankan	Alasan
Multi-sensor join (suhu + getaran)	Resample ke laju sama + Z-score	Skala fisik berbeda, distribusi tidak diketahui
Dashboard monitoring real-time	Downsample (mean) ke 1 Hz	Hemat bandwidth, tidak butuh resolusi tinggi
Training neural network	Min-max scaling [0,1]	Konvergensi lebih cepat, gradient lebih stabil
FFT/spectrum analysis	Resample uniform + jangan normalisasi	FFT butuh sampling uniform; normalisasi mengubah amplitudo asli
Anomaly detection berbasis statistik	Z-score per-window (rolling)	Adaptif terhadap perubahan baseline (drift)

Downsampling untuk Join Multi-Sensor: Downsampling multi-sensor perlu dilakukan hati-hati saat sensor-sensor tersebut akan digabungkan (joined) untuk analisis korelasi: menyamakan seluruh sensor ke sampling rate terendah di antara mereka (mis. suhu 1 Hz vs getaran 1000 Hz disamakan ke 1 Hz) memang menyederhanakan join, tetapi membuang informasi frekuensi tinggi getaran yang mungkin justru relevan. Alternatif yang lebih baik: hitung agregat statistik (RMS, peak, kurtosis) dari sinyal getaran resolusi penuh pada tiap jendela waktu yang sesuai dengan sampling rate sensor suhu, sehingga informasi frekuensi tinggi tidak hilang begitu saja melainkan terkondensasi menjadi fitur yang tetap bermakna secara statistik. Skemanya diperlihatkan pada Gambar 6.

Gambar 6. Ilustrasi downsampling ($f_s=20$ Hz, dekat batas Nyquist) dan upsampling via interpolasi pada sinyal 8 Hz.



Gambar 7. Distribusi data sebelum (raw) dan sesudah normalisasi Min-Max serta Z-score.

Perhatikan pada Gambar 7 bahwa bentuk distribusi (skewness, kurtosis relatif) tidak berubah setelah normalisasi, hanya rentang nilainya yang berbeda: Min-Max menghasilkan rentang [0,1] sedangkan Z-score menghasilkan mean nol dengan sebaran simetris di sekitarnya.

Validasi akhir wajib: Validasi akhir sebelum data dianggap siap dianalisis mencakup empat pemeriksaan: (1) plot perbandingan raw vs cleaned; (2) bandingkan statistik dasar sebelum dan sesudah; (3) pastikan `df.isnull().sum()` bernilai nol; (4) pastikan indeks waktu monoton naik tanpa duplikat.

IMPLEMENTASI PYTHON DI JUPYTER NOTEBOOK

Empat cell berikut menerapkan konsep pertemuan ini pada data sintesis yang meniru sensor accelerometer pada spindle mesin CNC milling. Lingkungan: conda env optoauto dengan paket numpy, pandas, scipy, matplotlib; notebook preprocessing_dasar.ipynb. Gambar 8 menunjukkan potongan Cell 3 yang mendeteksi outlier dengan Z-score dan IQR sekaligus.

```
preprocessing_dasar.ipynb

z = (y - np.nanmean(y)) / np.nanstd(y)
outlier = np.abs(z) > 3

q1, q3 = np.nanpercentile(y, [25, 75])
iqr = q3 - q1
lo, hi = q1 - 1.5*iqr, q3 + 1.5*iqr

y_capped = np.clip(y, lo, hi)

print(f'Outlier (Z): {outlier.sum()}')
print(f'Outlier (IQR): {(y<lo)|(y>hi)}.sum()')
```

Gambar 8. Potongan Cell 3: deteksi outlier Z-score dan IQR sekaligus, dilanjutkan dengan capping nilai ekstrem.

- **Cell 1:** generate sinyal sintesis ($f_s=10$ Hz, durasi 120 detik), trend+osilasi+noise, injeksi sekitar 5% missing dan 1% outlier (spike ± 8).
- **Cell 2:** bandingkan 3 metode imputasi (forward-fill, interpolasi time, mean fill), plot 3 subplot, cetak std masing-masing.
- **Cell 3:** deteksi outlier Z-score dan IQR pada hasil interpolasi, capping ke batas IQR (`np.clip`), plot sebelum dan sesudah capping.
- **Cell 4:** pipeline lengkap: Savitzky-Golay smoothing (window=21, polyorder=3), resample ke 1 Hz, min-max normalize [0,1], plot 2 subplot.

TUGAS PERTEMUAN 4

Tugas terdiri atas 10 soal Pilihan Ganda (1 poin), 10 soal Komputasi Easy-Medium (2 poin), dan 5 soal Komputasi Hard (4 poin), total 50 poin. Gambar 9 merangkum distribusi bobotnya.

Distribusi Bobot Tugas Pertemuan 4



Gambar 9. Distribusi 50 poin Tugas Pertemuan 4 berdasarkan tipe soal.

Bagian A: Pilihan Ganda (10 soal × 1 poin)

1. Sebuah sensor suhu pada motor mengalami gap missing pendek 1-3 sampel akibat noise komunikasi sesaat. Teknik imputasi mana yang paling tepat dan aman untuk situasi ini?

- A. Drop semua baris yang mengandung NaN, kehilangan kontinuitas waktu
- B. Interpolasi linear/forward-fill, mempertahankan kontinuitas dan mendekati nilai sebenarnya
- C. Isi dengan mean seluruh kolom, menumpulkan dinamika lokal
- D. Biarkan NaN dan teruskan ke model, model akan error

2. Sensor accelerometer pada bearing pompa sentrifugal menunjukkan spike amplitudo besar yang berulang dengan periode tetap. Mengapa membuang spike ini sebagai outlier Z-score bisa berbahaya untuk monitoring kondisi mesin?

- A. Karena algoritma deteksi outlier menjadi lambat
- B. Spike berulang adalah impulse dari bearing yang mulai retak, sinyal kerusakan dini, bukan noise
- C. Karena membuat distribusi tidak Gaussian
- D. Karena membuat standar deviasi data berkurang signifikan

3. Untuk dataset dengan distribusi non-Gaussian dan banyak outlier ekstrem, metode mana yang paling robust untuk mendeteksi outlier?

- A. Z-score standar ($|z| > 3$), karena mean dan std terdistorsi oleh outlier itu sendiri
- B. IQR ($Q_1 - 1,5 \cdot \text{IQR}$, $Q_3 + 1,5 \cdot \text{IQR}$), bebas asumsi distribusi, tidak terdistorsi
- C. Mean $\pm 2\sigma$, sama sensitifnya dengan Z-score
- D. Min/Max scaling, bukan metode deteksi outlier

4. Apa konsekuensi memilih window size yang terlalu besar pada moving average untuk smoothing sinyal vibrasi?

- A. Sinyal hasil menjadi lebih noisy daripada sinyal mentah
 - B. Tidak ada konsekuensi, semakin besar window selalu lebih baik
 - C. Puncak/transien tertumpul atau terhapus, indikator kerusakan dini bisa hilang
 - D. Kode menjadi error karena window melebihi panjang data
5. Pada resampling time series dari 100 Hz menjadi 10 Hz menggunakan rata-rata blok, syarat Nyquist criterion agar tidak terjadi aliasing adalah...
- A. Sinyal harus memiliki rata-rata nol
 - B. Window resampling harus berukuran ganjil
 - C. Frekuensi tertinggi sinyal harus di bawah 5 Hz, yaitu setengah dari sampling rate baru
 - D. Harus menggunakan interpolasi cubic spline
6. Saat membersihkan data sensor, ditemukan banyak baris dengan timestamp identik (duplikat). Pendekatan terbaik untuk menanganinya adalah...
- A. Biarkan saja, duplikat tidak memengaruhi analisis time series
 - B. Hapus dengan `df.drop_duplicates()` atau agregasi (mean/last) per timestamp
 - C. Tambahkan 1 milidetik ke setiap timestamp duplikat agar unik
 - D. Konversi indeks waktu menjadi string untuk mengabaikan duplikasi
7. Untuk dataset dengan beberapa outlier ekstrem, mengapa Modified Z-score (berbasis MAD) lebih disarankan daripada Z-score standar?
- A. Modified Z-score lebih cepat dihitung secara komputasi
 - B. Median dan MAD (median absolute deviation) tidak terdistorsi oleh outlier itu sendiri
 - C. Modified Z-score selalu menghasilkan nilai threshold $|z| > 2$
 - D. MAD memerlukan asumsi distribusi normal yang ketat
8. Saat downsampling sensor 1 kHz menjadi 10 Hz menggunakan agregasi rata-rata blok 100 sampel, efek apa yang akan terjadi pada sinyal?
- A. Memperbesar amplitudo spike pendek karena akumulasi
 - B. Spike pendek tertumpul dan baseline level tetap, tetapi komponen frekuensi tinggi hilang
 - C. Pasti menyebabkan aliasing tanpa peduli isi sinyal
 - D. Membuat seluruh hasil menjadi NaN karena pembagian nol
9. Rolling Z-score (Z-score dihitung dalam window bergerak) lebih unggul dari Z-score global untuk time series dengan karakteristik...
- A. Distribusi Gaussian sempurna dan stasioner
 - B. Tren/drift/perubahan baseline lambat, adaptif terhadap kondisi lokal
 - C. Dataset sangat kecil (< 100 sampel)

- D. Hanya satu kolom data tanpa indeks waktu

10. Min-Max Scaling [0,1] lebih disarankan dibanding Z-score Standardization ketika...

- A. Algoritma butuh batas terdefinisi, misalnya k-NN atau neural network dengan aktivasi sigmoid
- B. Distribusi data sangat skewed dengan outlier ekstrem
- C. Banyak outlier yang akan mengompresi rentang fitur lain
- D. Dataset memiliki jutaan baris dan butuh komputasi cepat

Bagian B: Komputasi Easy-Medium (10 soal × 2 poin)

C1. Sebuah sensor merekam dengan laju 10 Hz selama 120 detik. Hitung jumlah baris data yang dihasilkan (tidak termasuk header).

- 💡 **HINT:** Jumlah baris $N = \text{laju sampling} \times \text{durasi}$. Substitusikan nilai dari soal.

C2. Hitung jumlah bins histogram untuk dataset $N=1500$ sampel menggunakan aturan Sturges: $\text{bins} = \text{ceil}(\log_2(N) + 1)$.

- 💡 **HINT:** Aturan Sturges: $\text{bins} = \text{ceil}(\log_2(N) + 1)$. Pakai `np.log2` dan `np.ceil`, lalu bulatkan ke integer.

C3. Sebuah sensor memiliki distribusi nilai dengan mean $\mu=2,5$ dan std $\sigma=1,2$. Hitung Z-score untuk pembacaan ekstrem $x=8,5$.

- 💡 **HINT:** $Z\text{-score} = (x - \text{mean}) / \text{std}$. Substitusikan x , mean, dan std dari soal.

C4. Diberi data sensor [10,12,11,14,100,13,11,12,14,13]. Hitung upper bound IQR untuk deteksi outlier: $\text{upper} = Q_3 + 1,5 \times \text{IQR}$.

- 💡 **HINT:** Cari Q_1 dan Q_3 dengan `np.percentile`, lalu $\text{IQR} = Q_3 - Q_1$ dan $\text{upper bound} = Q_3 + 1,5 \times \text{IQR}$.

C5. Diberi sinyal [1,2,3,4,5,6,7,8,9,10]. Hitung rolling mean dengan window 5 menggunakan pandas dan ambil nilai rolling mean di indeks terakhir.

- 💡 **HINT:** Gunakan `Series.rolling(window=5).mean()` lalu ambil nilai di indeks terakhir dengan `.iloc[-1]`.

C6. Sebuah dataset memiliki 1500 baris dan 75 baris mengandung NaN. Hitung persentase missing (dalam %).

- 💡 **HINT:** $\text{Persentase missing} = (\text{jumlah baris NaN} / \text{total baris}) \times 100$. Substitusikan angka dari soal.

C7. Saat resampling sinyal dari 100 Hz menjadi 4 Hz, hitung frekuensi Nyquist (Hz) pada laju sampling baru.

- 💡 **HINT:** $\text{Frekuensi Nyquist} = f_{\text{sbaru}} / 2$. Substitusikan laju sampling baru dari soal.

C8. Diberi dataset suhu dengan $\text{min}=10\text{C}$ dan $\text{max}=50\text{C}$. Lakukan min-max scaling (normalisasi ke $[0,1]$) untuk nilai $x=35\text{C}$.

-  **HINT:** Min-max scaling = $(x - \text{min}) / (\text{max} - \text{min})$. Substitusikan x , min , dan max dari soal.

C9. Pada Z-score standardization dataset dengan $\text{mean}=20$ dan $\text{std}=5$, berapa nilai standardized untuk pembacaan $x=27,5$?

-  **HINT:** Standardisasi = $(x - \mu) / \sigma$. Substitusikan x , mean , dan std dari soal.

C10. Diberi sinyal $[2,4,6,\text{NaN},\text{NaN},12,14]$. Lakukan interpolasi linear menggunakan pandas, kemudian ambil nilai pada indeks ke-4 (NaN kedua) setelah interpolasi.

-  **HINT:** Buat `pd.Series` dari data, panggil `.interpolate()` untuk mengisi NaN , lalu ambil nilai pada indeks yang diminta dengan `.iloc`.

Bagian C: Komputasi Hard (5 soal × 4 poin)

Setiap soal membutuhkan algoritma lengkap ditulis dari awal, 20-50+ baris kode, hint minimal. Skema skor: jawaban benar = +4 poin, kode ada tapi hasil salah/error = +1 poin partial, kode kosong = 0 poin (bisa retry).

C11 (Hard). Implementasikan deteksi outlier Modified Z-score berbasis MAD (Median Absolute Deviation) tanpa `scipy.stats`. Data: $[12.1, 12.3, 12.0, 12.2, 45.0, 12.1, 12.4, 12.2, -8.0, 12.0, 12.1]$. Formula: $\text{MAD} = \text{median}(|x_i - \text{median}(x)|)$; $z_{\text{mod}} = 0,6745x(x_i - \text{median}(x))/\text{MAD}$; outlier jika $|z_{\text{mod}}| > 3,5$. Hitung jumlah outlier.

-  **HINT:** Hitung median data, lalu $\text{MAD} = \text{median}$ dari $|x_i - \text{median}|$; tiap $z_{\text{mod}} = 0,6745x(x_i - \text{median})/\text{MAD}$, lalu hitung berapa yang $|z_{\text{mod}}| > 3,5$.

C12 (Hard). Implementasikan Hampel Filter (rolling MAD outlier detection) dengan `window=5` (centered) dan `threshold=3x1,4826xMAD_lokal`. Data: $[5.1, 5.2, 5.0, 5.3, 25.0, 5.2, 5.1, 5.3, 5.0, -10.0, 5.2, 5.1, 5.3, 5.2, 5.0]$. Cek hanya indeks dengan window penuh. Hitung total outlier terdeteksi.

-  **HINT:** Loop tiap titik dengan window penuh; ambil window 5 titik, hitung median dan MAD lokal, lalu flag jika $|x_i - \text{median_lokal}|$ melebihi $3x1,4826x\text{MAD_lokal}$.

C13 (Hard). Implementasikan interpolasi linear manual (tanpa `pd.Series.interpolate()` atau `np.interp()`) untuk mengisi NaN . Data: $[10.0, 11.0, \text{NaN}, \text{NaN}, 14.0, 15.0, \text{NaN}, 17.0, 18.0]$. Formula: $\text{value}[i] = \text{value}[i_{\text{prev}}] + (i - i_{\text{prev}}) / (i_{\text{next}} - i_{\text{prev}}) \times (\text{value}[i_{\text{next}}] - \text{value}[i_{\text{prev}}])$. Hitung `sum()` dari array hasil interpolasi.

-  **HINT:** Untuk tiap NaN , cari indeks valid sebelum (i_{prev}) dan sesudah (i_{next}), lalu terapkan rumus interpolasi linear pada soal; akhirnya jumlahkan array dengan `sum()`.

C14 (Hard). Implementasikan Exponential Moving Average (EMA) dari awal (tanpa `pd.Series.ewm()`). Data: [10.0,12.0,11.0,13.0,14.0,13.0,15.0], $\alpha=0,3$. $EMA_0=data_0$; $EMA_i = \alpha \times data_i + (1-\alpha) \times EMA_{i-1}$ untuk $i \geq 1$. Cetak nilai EMA terakhir, dibulatkan 2 desimal.

-  **HINT:** Inisialisasi EMA dengan titik pertama, lalu iterasi $EMA_i = \alpha \times data_i + (1-\alpha) \times EMA_{i-1}$; ambil nilai EMA terakhir dan bulatkan 2 desimal.

C15 (Hard). Implementasikan pipeline 3-tahap deteksi + replacement outlier dengan IQR pada data: [5.0,5.1,5.2,50.0,5.3,5.4,5.5,-20.0,5.6,5.7,5.8,5.9,100.0,6.0,6.1]. Tahap: (1) hitung $Q_1/Q_3/IQR$ dan identifikasi outlier, (2) ganti outlier dengan median data bersih (median setelah outlier dikeluarkan), (3) hitung mean array hasil replacement, dibulatkan 2 desimal.

-  **HINT:** Tahap 1: cari Q_1/Q_3 dengan `np.percentile` dan batas $Q_1-1,5 \times IQR$ / $Q_3+1,5 \times IQR$; Tahap 2: ganti outlier dengan median data bersih; Tahap 3: hitung mean array hasil dan bulatkan 2 desimal.

DAFTAR PUSTAKA

- Dudek, G. (2023). STD: A seasonal-trend-dispersion decomposition of time series. *IEEE Transactions on Knowledge and Data Engineering*, 35(10), 10339–10350. <https://doi.org/10.1109/TKDE.2023.3268125>
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>
- Khan, S., & Lee, D.-H. (2017). An adaptive dynamically weighted median filter for impulse noise removal. *EURASIP Journal on Advances in Signal Processing*, 2017, 67. <https://doi.org/10.1186/s13634-017-0502-z>
- Romanssini, M., de Aguirre, P. C. C., Compassi-Severo, L., & Girardi, A. G. (2023). A review on vibration monitoring techniques for predictive maintenance of rotating machinery. *Eng*, 4(3), 1797–1817. <https://doi.org/10.3390/eng4030102>
- Ul Hassan, I., Panduru, K., & Walsh, J. (2024). An in-depth study of vibration sensors for condition monitoring. *Sensors*, 24(3), 740. <https://doi.org/10.3390/s24030740>
- Zhang, M., Guo, J., Li, X., & Jin, R. (2020). Data-driven anomaly detection approach for time-series streaming data. *Sensors*, 20(19), 5646. <https://doi.org/10.3390/s20195646>